Real-Time Chain-Snatching Detection System: Complete Implementation Roadmap

Executive Summary

This comprehensive roadmap provides step-by-step instructions for implementing a production-ready chain-snatching detection system. The project is divided into 4 phases: (1) Foundational Perception Layer with tracking, (2) Kinematic MVP with anomaly detection, (3) Deep Learning Verification, and (4) Optimization and deployment. Each phase includes concrete code examples, specific datasets from your research PDF, implementation details, and collaborative GitHub workflows for team development.

PHASE 1: Foundational Layer - Robust Tracking and Data Acquisition

Goal: Build a stable perception pipeline that can track all individuals with persistent IDs.

STEP 1.1: Setup Your Development Environment

What to do:

1. Install Python 3.8 or higher
2. Create a virtual environment:

```
python -m venv chainsnatch_env
source chainsnatch_env/bin/activate  # On Windows: chainsnatch_env\Scripts\activate
```

3. Install core libraries:

```
pip install ultralytics opencv-python numpy torch torchvision
```

Why this helps: Creates an isolated environment so dependencies don't conflict.

Output: A clean Python environment ready for development.

STEP 1.2: Download and Use Pre-trained YOLOv8 Model

What to do:

1. You **DO NOT need to train YOLOv8 from scratch** because it already comes pre-trained on COCO dataset which includes the 'person' class
2. Download pre-trained weights:

```
from ultralytics import YOLO

# This automatically downloads yolov8n.pt pre-trained on COCO
model = YOLO('yolov8n.pt')
```

Which dataset was used: COCO (Common Objects in Context) - 330K images with 'person' annotations.

Why this helps: The COCO pre-trained model already knows how to detect persons accurately. You're leveraging transfer learning, saving weeks of training time.

Output: A `yolov8n.pt` file that can detect persons in images/videos.

How to use this output: This model will feed detections into ByteTrack in the next step.

STEP 1.3: Fine-tune YOLOv8 on Crowded Scenes (Optional but Recommended)

What to do:

1. Download **CrowdHuman** dataset from: https://www.crowdhuman.org/
   - Contains 15K training images
   - Average 23 people per image
   - Has heavy occlusion (people blocking each other)
2. Download **CityPersons** dataset from: https://github.com/cvgroup-njust/CityPersons
   - Contains ~3K urban street scenes
   - Pedestrians in realistic surveillance-like conditions
3. Combine both datasets and create a YAML config file (`crowded_person.yaml`):

```
path: /path/to/combined_dataset/
train: 'train/images'
val: 'valid/images'

names:
  0: 'person'
```

4. Fine-tune YOLOv8:

```
from ultralytics import YOLO

model = YOLO('yolov8n.pt')  # Load COCO pre-trained model
results = model.train(
    data='crowded_person.yaml',
    epochs=50,
    imgsz=640,
    batch=8,
    name='yolov8n_crowded'
)
```

Why these datasets help:

- CrowdHuman teaches the model to detect people even when they're packed together and partially hidden
- CityPersons teaches detection in urban surveillance conditions similar to where chain-snatching occurs
- Together, they make your detector **much more robust** for crowded public spaces

Output: Fine-tuned weights saved at `runs/detect/yolov8n_crowded/weights/best.pt`

How to use this output: This fine-tuned model replaces the generic `yolov8n.pt` for better performance in your target environment.

STEP 1.4: Setup Video Ingestion Pipeline

What to do:

1. Create a Python script `video_ingestion.py`:

```
import cv2
from ultralytics import YOLO

# Load your model (fine-tuned or pre-trained)
model = YOLO('runs/detect/yolov8n_crowded/weights/best.pt')

# Open video stream (RTSP or local file)
cap = cv2.VideoCapture('rtsp://camera_ip:port/stream')  # For live camera
# OR
cap = cv2.VideoCapture('test_video.mp4')  # For local video

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    # Run detection
    results = model(frame)

    # Visualize
    annotated_frame = results[0].plot()
    cv2.imshow('YOLOv8 Detection', annotated_frame)

    if cv2.waitKey(1) &amp;amp; 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Why this helps: Tests that YOLOv8 can process real video streams and detect persons.

Output: A video player showing bounding boxes around detected persons in real-time.

STEP 1.5: Integrate ByteTrack for Multi-Object Tracking

What to do:

1. Install supervision library (includes ByteTrack):

```
pip install supervision
```

2. Create `tracking_pipeline.py`:

```
import cv2
import supervision as sv
from ultralytics import YOLO

# Load model
model = YOLO('runs/detect/yolov8n_crowded/weights/best.pt')

# Initialize ByteTrack
byte_tracker = sv.ByteTrack()

# Annotators for visualization
box_annotator = sv.BoxAnnotator()
label_annotator = sv.LabelAnnotator()
```

```
# Open video
cap = cv2.VideoCapture('test_video.mp4')

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    # Detect persons
    results = model(frame)[0]

    # Convert to supervision format
    detections = sv.Detections.from_ultralytics(results)

    # Filter for 'person' class only (class_id = 0 in COCO)
    detections = detections[detections.class_id == 0]

    # Update tracker
    detections = byte_tracker.update_with_detections(detections)

    # Prepare labels showing track IDs
    labels = [f"ID: {tracker_id}" for tracker_id in detections.tracker_id]

    # Annotate frame
    annotated_frame = box_annotator.annotate(frame.copy(), detections=detections)
    annotated_frame = label_annotator.annotate(annotated_frame, detections=detections, labe

    cv2.imshow('YOLOv8 + ByteTrack', annotated_frame)

    if cv2.waitKey(1) &amp;amp; 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Why this helps: ByteTrack assigns persistent IDs to each person, maintaining their identity across frames even during occlusions.

Output: Video showing bounding boxes with **Track IDs** (e.g., "ID: 1", "ID: 2", etc.) that persist as people move.

Critical insight: These Track IDs are the **foundation** of your entire system. Every subsequent analysis (optical flow, force calculations, pose estimation) will be organized by these IDs.

STEP 1.6: Benchmark Tracking Performance on MOTChallenge

What to do:

1. Download **MOT17** or **MOT20** dataset from: https://motchallenge.net/
   - These contain ground truth tracking annotations
   - Used to measure tracking accuracy
2. Run your YOLOv8 + ByteTrack pipeline on MOT sequences
3. Evaluate using official MOTChallenge metrics:

```
git clone https://github.com/dendorferpatrick/MOTChallengeEvalKit
cd MOTChallengeEvalKit
python -m motmetrics.apps.eval_motchallenge path/to/groundtruth path/to/predictions
```

Why this helps: Gives you objective metrics (MOTA, IDF1) to know if your tracking is working well before building on top of it.

Output: Numerical scores like:

- MOTA (Multiple Object Tracking Accuracy): ~70%
- IDF1 (ID F1 Score): Measures how well IDs are maintained

STEP 1.7: Parallel Data Acquisition (Ongoing Throughout Project)

What to do - START THIS NOW:

1. **Download UCF-Crime dataset**: https://www.crcv.ucf.edu/projects/real-world/
   - Contains videos labeled "Robbery", "Stealing", "Assault"
   - These are **proxy events** similar to chain-snatching
   - Use these for training your final anomaly classifier in Phase 3
2. **Download Avenue dataset**: https://www.cse.cuhk.edu.hk/leojia/projects/detectabnormal/dataset.html
   - Contains normal pedestrian behavior and general anomalies
   - Use for learning "normal" vs "unusual" motion patterns
3. **Start collecting/recording real footage**:
   - Record staged chain-snatching scenarios with actors
   - Search online for real chain-snatching CCTV footage
   - This custom data will be **critical** for final model performance

Why this helps: Data collection takes time. Starting early means you'll have quality data ready when you need it in Phase 3.

Output: A growing library of videos organized by:

- `normal_behavior/` - everyday pedestrian activity
- `anomalous_interactions/` - robberies, stealing, assaults, chain-snatching

PHASE 2: MVP - Kinematic Anomaly Detection

Goal: Build Tier 1 detector that flags physically anomalous interactions using forces.

STEP 2.1: Extract Optical Flow for Tracked Persons

What to do:

1. Modify your tracking script to calculate optical flow within each person's bounding box:

```
import cv2
import numpy as np
import supervision as sv
from ultralytics import YOLO

model = YOLO('runs/detect/yolov8n_crowded/weights/best.pt')
byte_tracker = sv.ByteTrack()

cap = cv2.VideoCapture('test_video.mp4')
ret, prev_frame = cap.read()
prev_gray = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY)

# Dictionary to store velocity for each track ID
track_velocities = {}

while cap.isOpened():
```

```
    ret, frame = cap.read()
    if not ret:
        break

    curr_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Detection and tracking
    results = model(frame)[0]
    detections = sv.Detections.from_ultralytics(results)
    detections = detections[detections.class_id == 0]
    detections = byte_tracker.update_with_detections(detections)

    # For each tracked person
    for i, (bbox, track_id) in enumerate(zip(detections.xyxy, detections.tracker_id)):
        x1, y1, x2, y2 = map(int, bbox)

        # Extract bounding box region
        prev_roi = prev_gray[y1:y2, x1:x2]
        curr_roi = curr_gray[y1:y2, x1:x2]

        # Calculate dense optical flow (Farneback method)
        flow = cv2.calcOpticalFlowFarneback(
            prev_roi, curr_roi, None,
            0.5, 3, 15, 3, 5, 1.2, 0
        )

        # Calculate average velocity
        vx = np.mean(flow[..., 0])
        vy = np.mean(flow[..., 1])

        track_velocities[track_id] = np.array([vx, vy])

        print(f"Track ID {track_id}: velocity = ({vx:.2f}, {vy:.2f})")

    prev_gray = curr_gray.copy()

cap.release()
```

Why this helps: Optical flow gives you the **velocity vector** of each person, which is required to calculate interaction forces.

Output: A dictionary `track_velocities` mapping each Track ID to its velocity vector `[vx, vy]`.

How to use this output: These velocities feed into the Social Force Model in the next step.

STEP 2.2: Implement Social Force Model

What to do:

1. Install the differentiable social force library:

```
pip install socialforce
```

2. Create `social_force_detector.py`:

```
import numpy as np

def calculate_repulsive_force(pos1, vel1, pos2, vel2):
    """
    Calculate repulsive social force between two persons

    Args:
```

```
        pos1, pos2: Position vectors [x, y]
        vel1, vel2: Velocity vectors [vx, vy]

    Returns:
        force_magnitude: Scalar repulsive force
    """
    # Distance between persons
    r = pos2 - pos1
    distance = np.linalg.norm(r)

    if distance &lt; 0.1:  # Avoid division by zero
        return 0

    # Direction vector
    direction = r / distance

    # Relative velocity
    relative_vel = vel1 - vel2

    # Social force parameters (tune these)
    A = 2.1  # Interaction strength
    B = 0.3  # Interaction range

    # Repulsive force magnitude (exponential decay with distance)
    force_magnitude = A * np.exp(-distance / B)

    # Additional factor: stronger if moving toward each other
    approach_factor = -np.dot(relative_vel, direction)
    if approach_factor &gt; 0:
        force_magnitude *= (1 + approach_factor)

    return force_magnitude
```

3. Integrate into your tracking pipeline:

```
# Inside your tracking loop, after getting track_velocities

track_positions = {}  # Store center positions

for i, (bbox, track_id) in enumerate(zip(detections.xyxy, detections.tracker_id)):
    x1, y1, x2, y2 = bbox
    center_x = (x1 + x2) / 2
    center_y = (y1 + y2) / 2
    track_positions[track_id] = np.array([center_x, center_y])

# Calculate forces between all pairs
force_events = []
track_ids = list(track_positions.keys())

for i in range(len(track_ids)):
    for j in range(i+1, len(track_ids)):
        id1, id2 = track_ids[i], track_ids[j]

        pos1 = track_positions[id1]
        pos2 = track_positions[id2]
        vel1 = track_velocities.get(id1, np.array([0, 0]))
        vel2 = track_velocities.get(id2, np.array([0, 0]))

        force = calculate_repulsive_force(pos1, vel1, pos2, vel2)

        # If force exceeds threshold, flag as candidate event
        FORCE_THRESHOLD = 5.0  # Tune this value
        if force &gt; FORCE_THRESHOLD:
            force_events.append({
                'track_id_1': id1,
```

```
                'track_id_2': id2,
                'force_magnitude': force,
                'frame_number': frame_count
            })
            print(f"⚠ ANOMALY DETECTED! Force={force:.2f} between ID {id1} and ID {id2}")
```

Why this helps: The Social Force Model quantifies when two people are interacting in a physically abnormal way (e.g., sudden approach with high force).

Output: A list of `force_events` containing:

- Which Track IDs were involved
- The force magnitude
- When it happened (frame number)

How to use this output: These flagged events become **triggers** for Phase 3's deep learning verification.

STEP 2.3: Threshold-Based Event Trigger

What to do:

1. Collect normal behavior videos and run your force calculation
2. Record the distribution of force magnitudes during normal interactions
3. Set threshold at 99th percentile:

```
import numpy as np

# Collect force magnitudes from normal videos
normal_forces = [...]  # Run system on normal videos, collect all forces

FORCE_THRESHOLD = np.percentile(normal_forces, 99)
print(f"Threshold set at: {FORCE_THRESHOLD}")
```

Why this helps: Ensures you flag only the most extreme 1% of interactions as anomalous.

Output: A calibrated threshold value.

PHASE 3: High-Fidelity Verification - Deep Learning Core

Goal: Add semantic verification to dramatically reduce false positives.

STEP 3.1: Setup RTMPose for Pose Estimation

What to do:

1. Install RTMLib (lightweight RTMPose):

```
pip install rtmlib
```

2. Create `pose_verification.py`:

```
from rtmlib import PoseTracker
import cv2

# Initialize pose tracker
pose_tracker = PoseTracker(
    det_frequency=5,  # Run detection every 5 frames
    mode='lightweight'
```

```
)

def get_pose_for_persons(frame, bounding_boxes):
    """
    Args:
        frame: Image frame
        bounding_boxes: List of [x1, y1, x2, y2] for each person

    Returns:
        keypoints: List of 17 keypoints per person
    """
    keypoints, scores = pose_tracker(frame, bboxes=bounding_boxes)
    return keypoints, scores
```

Which dataset was used: RTMPose models are pre-trained on **COCO-Pose** (200K+ annotated humans) and **MPII Human Pose** (25K images, 410 activities).

Why this helps: Pose keypoints let you detect specific actions like "hand reaching toward neck area" which is a strong indicator of chain-snatching.

Output: For each person, 17 keypoints: [nose, left_eye, right_eye, left_ear, right_ear, left_shoulder, right_shoulder, left_elbow, right_elbow, left_wrist, right_wrist, left_hip, right_hip, left_knee, right_knee, left_ankle, right_ankle]

STEP 3.2: Setup Mask R-CNN (On-Demand)

What to do:

1. Install Detectron2 (Facebook's Mask R-CNN implementation):

```
pip install 'git+https://github.com/facebookresearch/detectron2.git'
```

2. Download COCO pre-trained Mask R-CNN:

```
from detectron2 import model_zoo
from detectron2.config import get_cfg
from detectron2.engine import DefaultPredictor

cfg = get_cfg()
cfg.merge_from_file(model_zoo.get_config_file("COCO-InstanceSegmentation/mask_rcnn_R_50_FPI
cfg.MODEL.WEIGHTS = model_zoo.get_checkpoint_url("COCO-InstanceSegmentation/mask_rcnn_R_50_
cfg.MODEL.ROI_HEADS.SCORE_THRESH_TEST = 0.5

mask_rcnn_predictor = DefaultPredictor(cfg)

def get_segmentation_masks(frame, bounding_boxes):
    """Only called when force-based trigger fires"""
    outputs = mask_rcnn_predictor(frame)
    return outputs["instances"].pred_masks
```

Why this helps: Mask R-CNN provides pixel-level segmentation to detect fine-grained details like chains, hands, contact regions.

Critical note: Mask R-CNN is **only activated when Phase 2 flags an event**. It does NOT run continuously.

STEP 3.3: Train Contact Detection Network

This is the **most critical and custom component**.

What to do:

1. **Prepare training data** using UCF-Crime dataset:
   - Extract frames from "Robbery", "Stealing", "Assault" videos
   - Label frames showing hand-on-person contact as **POSITIVE**
   - Label normal interactions as **NEGATIVE**
2. **Use HICO-DET** interaction annotation methodology:
   - Label interactions as structured triplets: `<Subject, Action, Object>`
   - Example: `<Person_12, Grabs_Chain, Person_25>`
3. **Build a simple contact classifier**:

```
import torch
import torch.nn as nn

class ContactDetector(nn.Module):
    def __init__(self):
        super().__init__()
        # Input: concatenated features from two persons
        # - 17 keypoints × 2 persons × 2 coords = 68 dims
        # - 2 velocity vectors × 2 = 4 dims
        # - 1 force magnitude = 1 dim
        # Total: 73 input features

        self.fc1 = nn.Linear(73, 128)
        self.fc2 = nn.Linear(128, 64)
        self.fc3 = nn.Linear(64, 1)  # Binary: contact or no contact
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.sigmoid(self.fc3(x))
        return x

# Training loop
model = ContactDetector()
criterion = nn.BCELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# Load your labeled data here
# X_train: feature vectors (pose + velocity + force)
# y_train: labels (1 = contact event, 0 = no contact)

for epoch in range(100):
    optimizer.zero_grad()
    outputs = model(X_train)
    loss = criterion(outputs, y_train)
    loss.backward()
    optimizer.step()

torch.save(model.state_dict(), 'contact_detector.pth')
```

Which datasets to use:

- **UCF-Crime**: "Robbery", "Stealing", "Assault" videos for positive examples

- **Avenue dataset**: Normal pedestrian behavior for negative examples
- **Your custom staged footage**: Most valuable for actual chain-snatching patterns

Why this helps: This custom network learns the specific signature of malicious physical contact, going beyond what generic models can do.

Output: A trained `contact_detector.pth` model file.

STEP 3.4: Integrate All Verification Components

What to do - Create `integrated_system.py`:

```
import cv2
import numpy as np
import torch
import supervision as sv
from ultralytics import YOLO
from rtmlib import PoseTracker

# Load all models
yolo_model = YOLO('runs/detect/yolov8n_crowded/weights/best.pt')
byte_tracker = sv.ByteTrack()
pose_tracker = PoseTracker(mode='lightweight')
contact_model = ContactDetector()
contact_model.load_state_dict(torch.load('contact_detector.pth'))
contact_model.eval()

# Configuration
FORCE_THRESHOLD = 5.0
CONTACT_CONFIDENCE_THRESHOLD = 0.8

# Main loop
cap = cv2.VideoCapture('test_video.mp4')

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    # PHASE 1: Detection + Tracking
    results = yolo_model(frame)[0]
    detections = sv.Detections.from_ultralytics(results)
    detections = detections[detections.class_id == 0]
    detections = byte_tracker.update_with_detections(detections)

    # PHASE 2: Calculate forces (simplified)
    track_positions, track_velocities = extract_kinematics(detections, frame)
    force_events = calculate_social_forces(track_positions, track_velocities, FORCE_THRESH

    # PHASE 3: If force event detected, verify with deep learning
    for event in force_events:
        id1, id2 = event['track_id_1'], event['track_id_2']
        force = event['force_magnitude']

        # Get bounding boxes for the two individuals
        bbox1 = get_bbox_for_track(detections, id1)
        bbox2 = get_bbox_for_track(detections, id2)

        # Extract poses
        keypoints, _ = pose_tracker(frame, bboxes=[bbox1, bbox2])

        # Prepare feature vector
        features = np.concatenate([
```

```
            keypoints[0].flatten(),  # Person 1 pose
            keypoints[1].flatten(),  # Person 2 pose
            track_velocities[id1],
            track_velocities[id2],
            [force]
        ])

        # Run contact detection
        with torch.no_grad():
            features_tensor = torch.FloatTensor(features).unsqueeze(0)
            contact_prob = contact_model(features_tensor).item()

        # FINAL DECISION
        if contact_prob &gt; CONTACT_CONFIDENCE_THRESHOLD:
            print(f"🚨 VERIFIED CHAIN-SNATCHING EVENT!")
            print(f"   Force: {force:.2f}")
            print(f"   Contact Confidence: {contact_prob:.2%}")
            print(f"   Between Track ID {id1} and {id2}")

            # TRIGGER ALERT (send to security, save clip, etc.)
            trigger_alert(frame, event, contact_prob)

cap.release()
```

Output: A complete system that:

1. Detects and tracks all persons
2. Flags anomalous force interactions
3. Verifies with pose and contact detection
4. Only alerts on verified events

PHASE 4: Optimization and Deployment

Goal: Make the system production-ready with TensorRT optimization.

STEP 4.1: Export YOLOv8 to TensorRT

What to do:

1. Export YOLOv8 to TensorRT engine:

```
yolo export model=runs/detect/yolov8n_crowded/weights/best.pt format=engine device=0
```

2. Use TensorRT model in deployment:

```
model = YOLO('runs/detect/yolov8n_crowded/weights/best.engine')
```

Why this helps: TensorRT optimizes the model for NVIDIA GPUs, providing 2-4x speedup.

Output: A `.engine` file optimized for your specific GPU.

STEP 4.2: Quantize Models to INT8

What to do:

1. Follow TensorRT quantization guide to convert FP32 weights to INT8
2. This requires calibration data (a few hundred representative images)

Why this helps: INT8 quantization provides additional 2x speedup with minimal accuracy loss.

STEP 4.3: Deploy with NVIDIA DeepStream (Optional for Production)

What to do:

1. Install DeepStream SDK: https://developer.nvidia.com/deepstream-sdk
2. Convert your pipeline to DeepStream configuration
3. DeepStream manages multi-stream processing efficiently

Why this helps: DeepStream is designed for production-scale video analytics with features like automatic load balancing and hardware-accelerated decoding.

GitHub Collaboration Strategy

Repository Structure:

```
chain-snatching-detection/
├── phase1_perception/
│   ├── yolov8_training.py
│   ├── bytetrack_integration.py
│   └── models/
│       └── best.pt
├── phase2_kinematic/
│   ├── optical_flow.py
│   ├── social_force_model.py
│   └── config.yaml
├── phase3_verification/
│   ├── rtmpose_integration.py
│   ├── contact_detector.py
│   └── train_contact_model.py
├── phase4_deployment/
│   ├── tensorrt_export.sh
│   └── deepstream_config.txt
├── datasets/
│   ├── download_scripts/
│   └── README.md
├── integrated_system.py
├── requirements.txt
└── README.md
```

Team Workflow:

Team Member A (Perception Engineer):

1. Works in `phase1_perception/`
2. Responsible for YOLOv8 fine-tuning and ByteTrack integration
3. Outputs: `best.pt` model file and `track_ids.json`

Team Member B (Kinematic Engineer):

1. Works in `phase2_kinematic/`

2. Takes `track_ids.json` as input
3. Outputs: `force_events.json` (list of flagged interactions)

Team Member C (Deep Learning Engineer):

1. Works in `phase3_verification/`
2. Takes `force_events.json` as input
3. Outputs: `verified_events.json` (final alerts)

Connection Points (How to Share Outputs):

1. **Standardize data format** - Create `data_schemas.py`:

```
# Shared data format
class Detection:
    frame_id: int
    track_id: int
    bbox: List[float]  # [x1, y1, x2, y2]
    confidence: float

class ForceEvent:
    frame_id: int
    track_id_1: int
    track_id_2: int
    force_magnitude: float

class VerifiedEvent:
    frame_id: int
    track_id_1: int
    track_id_2: int
    force_magnitude: float
    contact_confidence: float
    alert: bool
```

2. **Use JSON files to pass data between phases**:

```
import json

# Team Member A saves tracking output
with open('outputs/track_ids.json', 'w') as f:
    json.dump(tracking_results, f)

# Team Member B loads it
with open('outputs/track_ids.json', 'r') as f:
    tracking_results = json.load(f)
```

3. **Git workflow**:

```
# Each team member works on their feature branch
git checkout -b feature/phase1-tracking

# Commit your work
git add .
git commit -m "Implemented ByteTrack integration"

# Push to shared repository
git push origin feature/phase1-tracking

# Create Pull Request for review
# After review, merge to main branch
```

4. **Integration testing**:

- Create `tests/integration_test.py` that runs end-to-end
- Each team member ensures their component works with test data
- Before merging, run full integration test

Summary Timeline

Phase	Duration	Deliverable
Phase 1	3-4 weeks	Working YOLOv8 + ByteTrack tracking system
Phase 2	2-3 weeks	MVP with force-based anomaly detection
Phase 3	6-8 weeks	Complete system with semantic verification
Phase 4	2-3 weeks	Production-optimized deployment

Total: 13-18 weeks for a complete system.